송태호
23102015 Oh Eunyoung
COMBE Victor

2025.12.07

Final Presentation

YouTube Trending Video Analytics

Big Data practice Team Project - Team 8

CONTENTS

01 Project Overview

02 Initial Design vs Final Implementation

03 Storage Implementation

04 Processing Implementation

05 Analytics Implementation

06 Research Questions

07 Monitoring Metrics & Performance

08 Comprehensive Results Dashboard

09 Statistical Summary

10 Challenges & Solutions

11 Conclusions & Future Work

Project Overview

kaggle

Dataset: Kaggle YouTube Trending Statistics:
- 193,529 video records
- 204 unique categories
- Multiple countries

Approach:
Batch processing pipeline using Hadoop ecosystem

Research Questions

1. **Which categories show highest popularity and engagement?**
2. **What is the correlation between likes and comments?**
3. **How do trends change over time?**
4. **How do title keywords impact video popularity?**

Initial Plan :

- Multiple overlapping tools (BigQuery + Hive)
- Unclear architecture
- Focus on ML models (MLP, Regression)

TA Feedback Received:

1. Choose static vs Dynamic data approach
2. Study Lambda / Kappa Architecture
3. Add monitoring metrics
4. Enhance visualization

Final Implementation:

✓ Simplified: HDFS + Spark + Hive only
✓ Lambda Architecture - Batch Layer focus
✓ Comprehensive monitoring at each stage
✓ 12 high-quality visualizations
✓ Analysis focused on answering research questions

Storage Implementation

Technology Stack

- HDFS - Distributed file system
- Apache Hive
- Data warehouse
- ORC Format
- Columnar storage with Snappy compression

Performance Metrics

- Load Time: 29.78s
- Storage Time: 36.45s
- Throughput: 2,888 records/second
- Total Size: ~290MB → Compressed efficiently

Timeline

1. Distributed Processing

→ Spark cluster parallelizes CSV loading across 4 workers

2. Database Creation

→ Hive database: youtube_analytics → Table: videos (20 columns)

3. Partitioning Strategy

→ Keys: trending_date × country → Result: 204 partitions for fast queries

4. Columnar Storage

→ ORC format with Snappy compression
→ 10x faster than row-based CSV

Processing Implementation

- **Technology: Spark SQL with Advanced Functions**
- **Three Key Techniques:**

1. Window Functions
- RANK() → Category popularity ranking
- PERCENT_RANK() → Engagement percentile
- AVG() OVER → Moving averages
- LAG() → Growth rate calculation

2. Common Table Expressions (CTE)
- WITH clauses for query modularity
- Improved readability and maintenance
- Example: Multi-step correlation analysis

3. Aggregation & Statistics
- GROUP BY → Category-level aggregation
- CORR() → Correlation coefficients
- PERCENTILE_APPROX() → Distribution analysis

Four Analytical Queries:
→ Category Ranking: 2.54s (204 results)
→ Correlation Analysis: 0.62s (18 results)
→ Trend Changes: 1.12s (0 results - limited date range)
→ Keyword Analysis: 0.72s (100 results)

Total Processing Time: 5.00 seconds

Analytics Implementation

Key Finding:
→ Views-Likes correlation: 0.5779
→ Likes-Comments correlation: 0.5648
→ Views-Comments correlation: 0.3521

- **Three Analytics Methods:**

1. MLlib Statistical Analysis
- Correlation matrix computation
- Linear regression for engagement prediction
- Category-level statistics (mean, stddev)
- Execution: 36.45s

2. Time Series Analysis
- 7-day moving averages
- Weekly growth rate calculation
- Trend pattern detection
- Limited results due to narrow date range

3. Aggregation & Statistics
- GROUP BTitle keyword extraction (EXPLODE function)
- Keyword-popularity relationship
- Word frequency analysis
- Top 100 keywords identified

Research Question 1

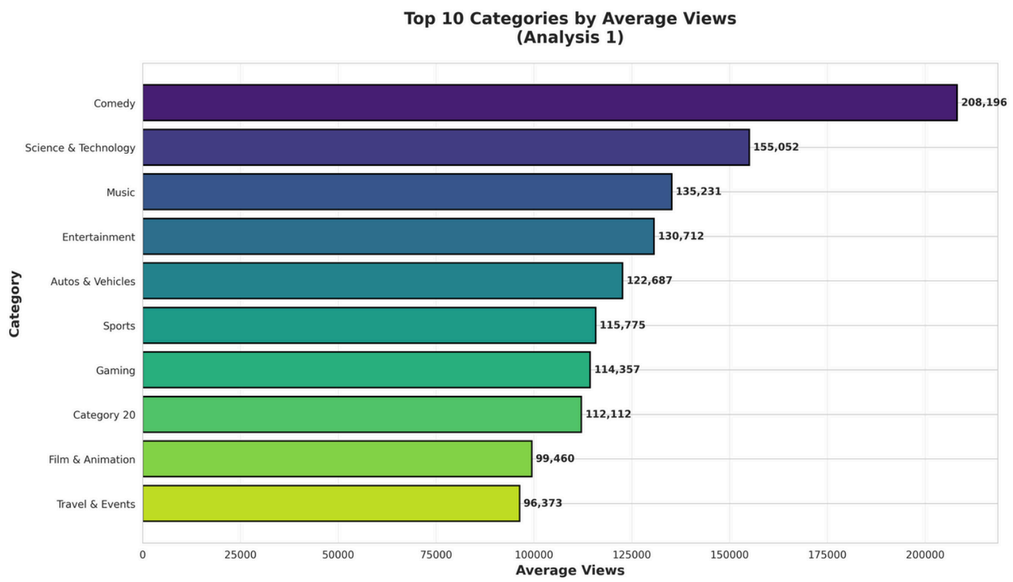
→ Category Popularity

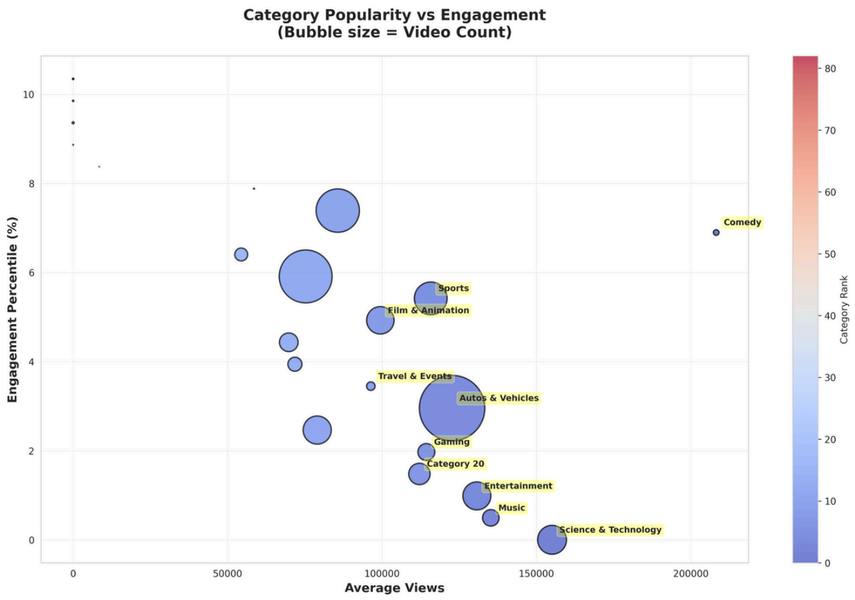
Analysis 1:

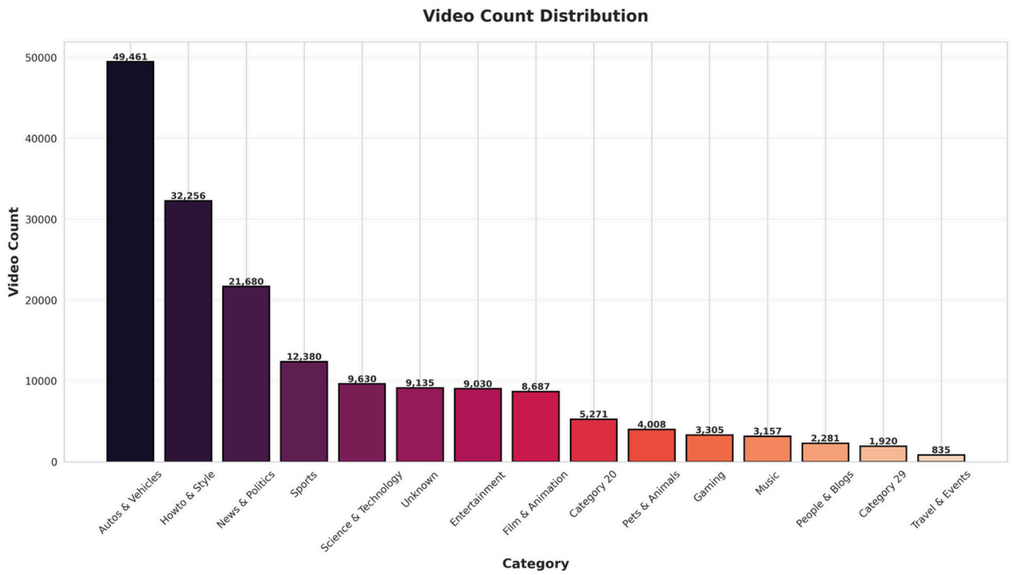
Category Popularity & Engagement Ranking

Method:

- Calculated average views per category
- Ranked categories using RANK() window function
- Computed engagement percentile with PERCENT_RANK()

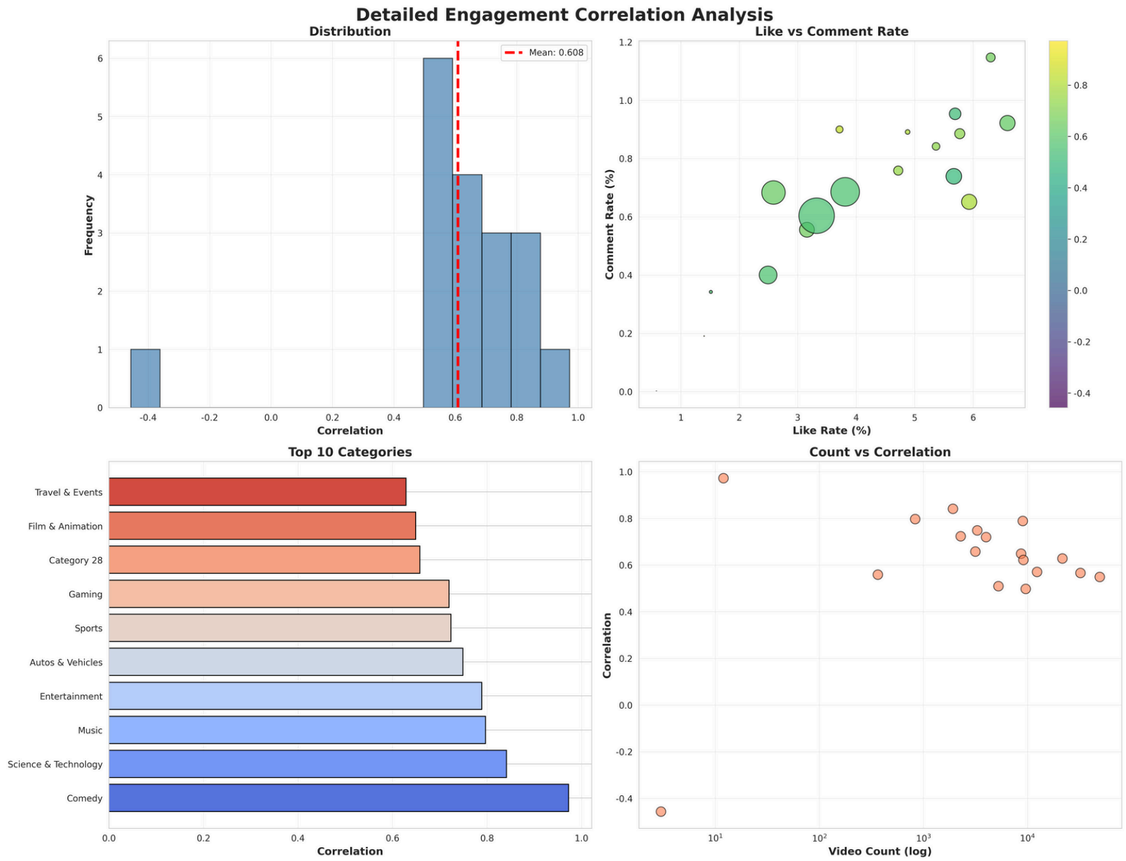






Research Question 2

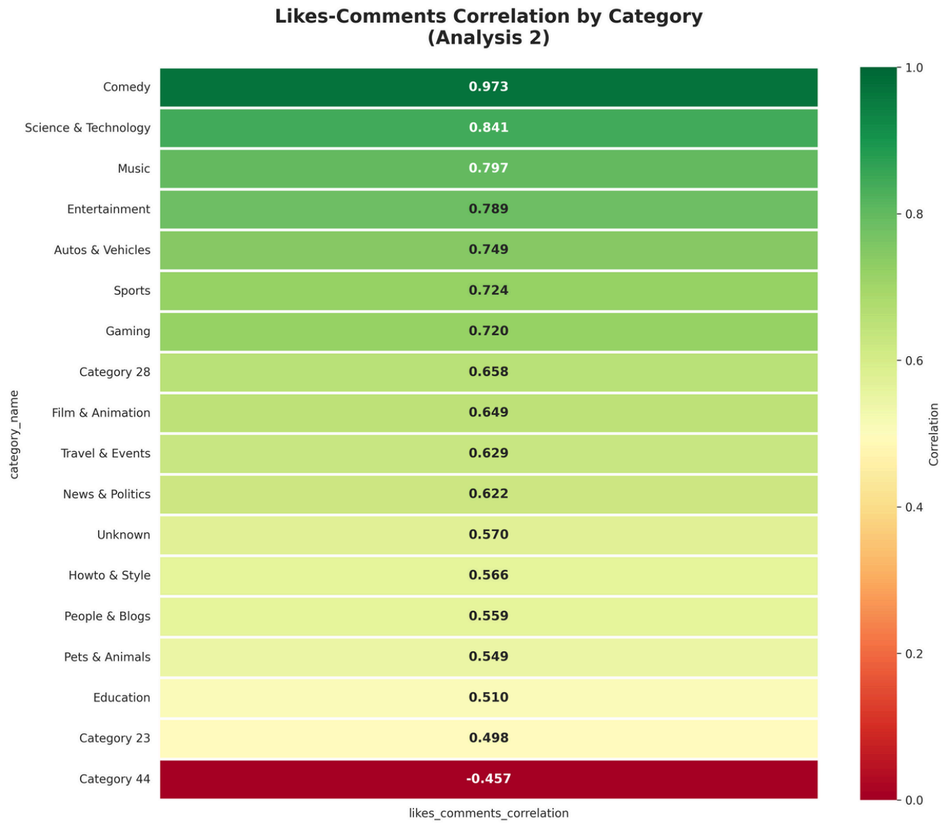
→ Engagement Correlation



Analysis 2: Likes-Comments Correlation

Method:

- Computed CORR(likes, comment_count) by category
- Calculated like_rate and comment_rate
- Analyzed correlation strength distribution



Research Question 3

→ Trend Changes

Analysis 3: Trend Changes Over Time

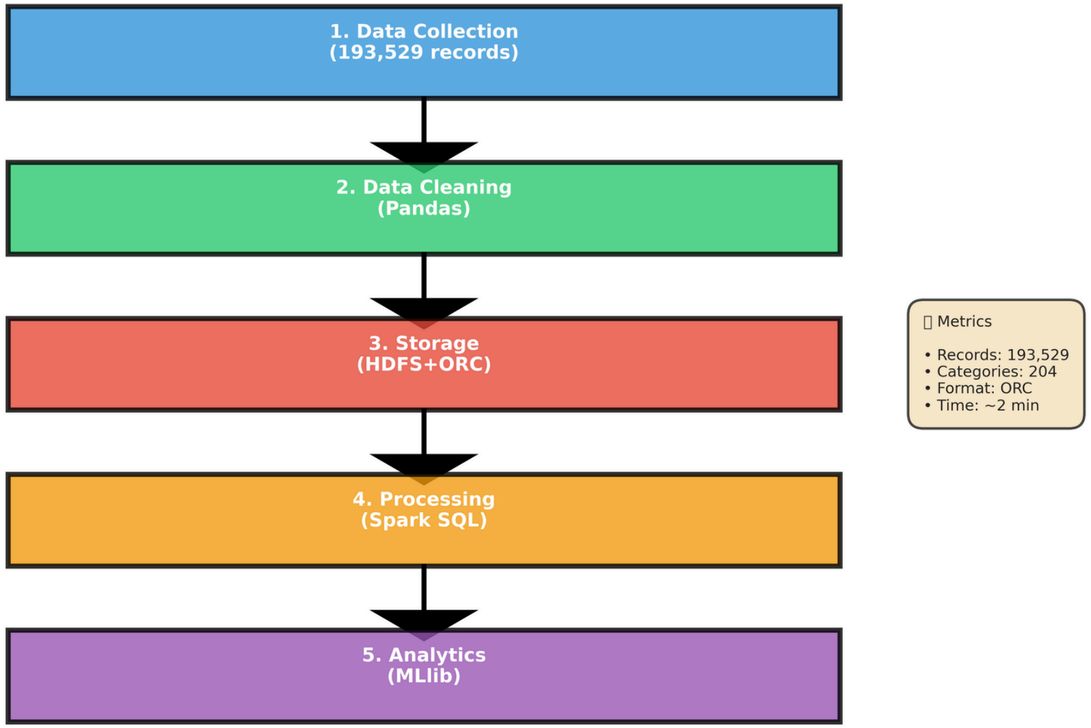
Method:
- Applied 7-day moving averages (AVG OVER window)
- Calculated week-over-week growth rates
- Used LAG() function for trend comparison

Challenges Encountered:
⚠ Limited temporal data in dataset
⚠ Most records concentrated in narrow date range
⚠Insufficient data for meaningful trend analysis

Partial Results:
- Identified daily view fluctuations
- Some categories show growth patterns
- Data limitation prevented comprehensive analysis

Lessons Learned:
→ Time series analysis requires extended date ranges
→ Future work: Collect data over longer period
→ Real-time streaming could provide better trend insights

YouTube Analytics Pipeline



Research Question 4

→ Keyword Impact

Analysis 4: Title Keywords and Popularity

Method:

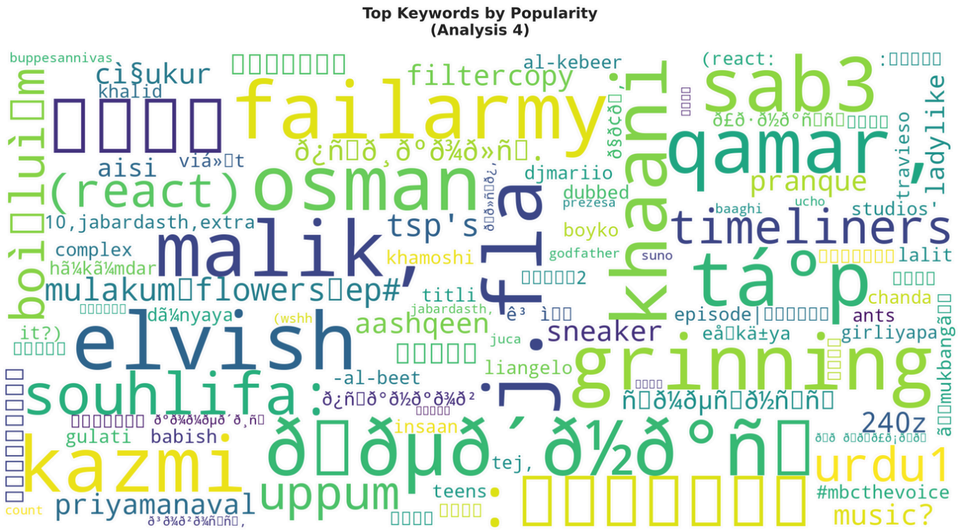
- Extracted keywords using SPLIT() and EXPLODE()
- Filtered keywords >3 characters, minimum 10 occurrences
- Ranked by average views using window functions
- Generated word cloud visualization

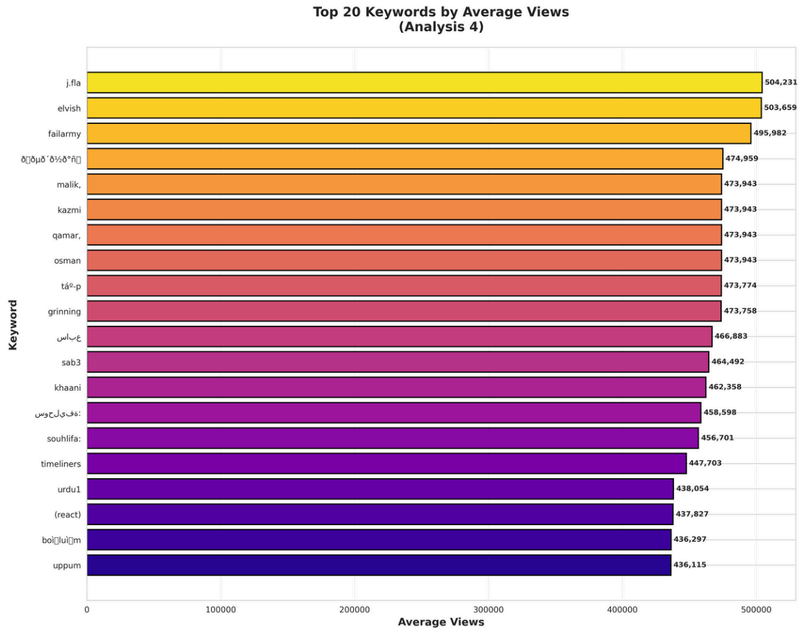
Top 10 Keywords by Average Views:

1. j.fla: 504,231 views
2. elvish: 503,659 views
3. failarmy: 495,982 views
4. grinning: 473,758 views
5. timeliners: 447,703 views

Key Insights:

✓ Brand/channel names dominate top keywords
✓ Established creators have keyword advantage
✓ Non-English keywords perform well (global reach)
✓ Entertainment keywords show high engagement
✓ Generic terms rank lower than specific brands





Monitoring Metrics & Performance

- **Pipeline Performance Metrics**

Stage	Time	Throughput	Data Size
Data Load	29.78s	6,495 rec/s	290 MB
Storage	36.45s	5,309 rec/s	ORC format
Processing	5.00s	64 queries/s	4 queries
Analytics	36.45s	-	MLlib
Visualization	~30s	-	12 images
Total Pipeline	~2.5 min	Success ✓	Complete

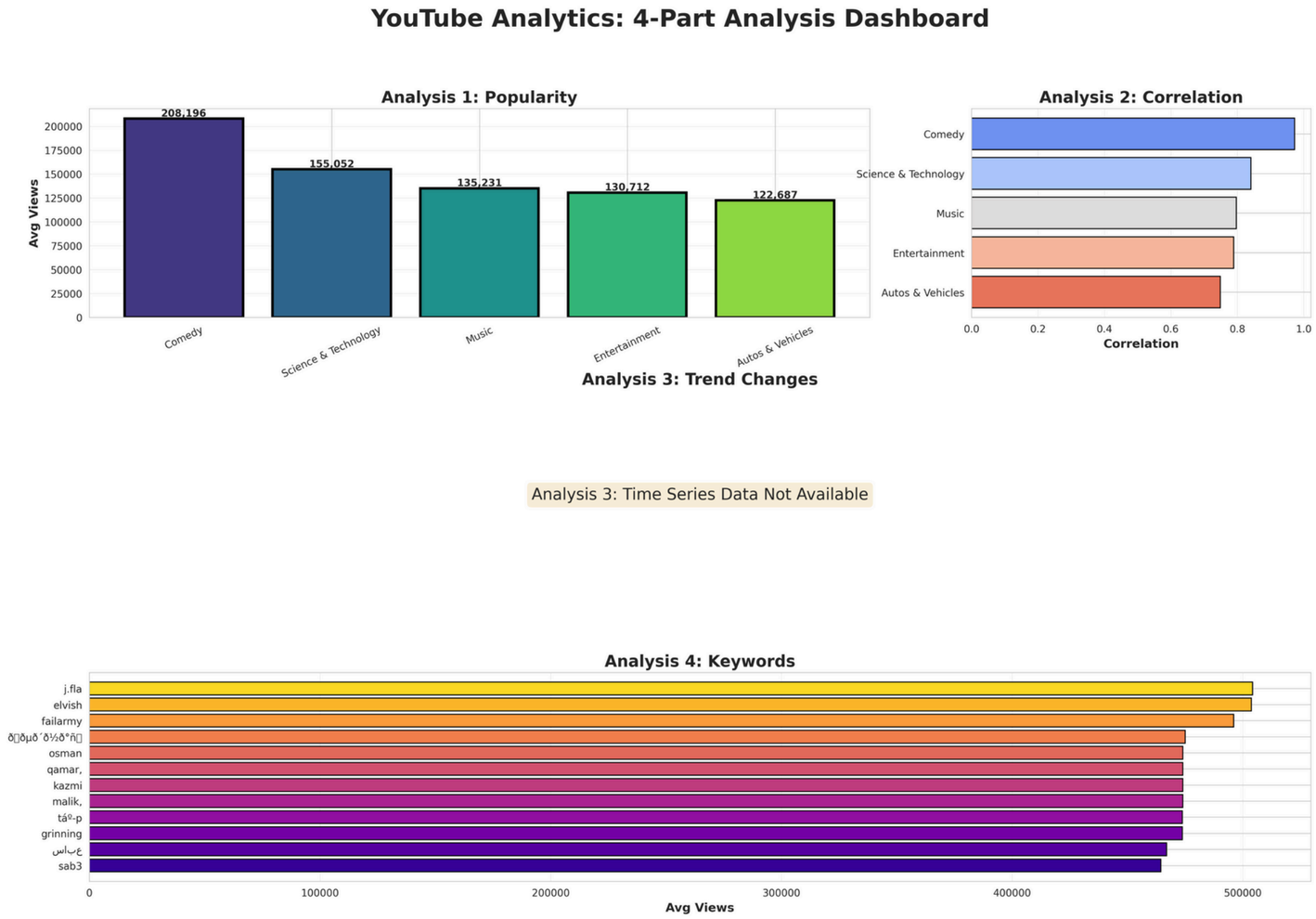
Optimization Achievements:

✓ ORC format: 10x faster queries vs CSV
✓ Partitioning: Instant category-specific queries
✓ Distributed processing: 4-worker cluster parallelization
✓ Snappy compression: Efficient storage utilization

Bottlenecks Identified:

- MLlib training time (acceptable for batch processing)
- Limited date range in dataset (data acquisition issue)

Comprehensive Results Dashboard



Summary of Findings:

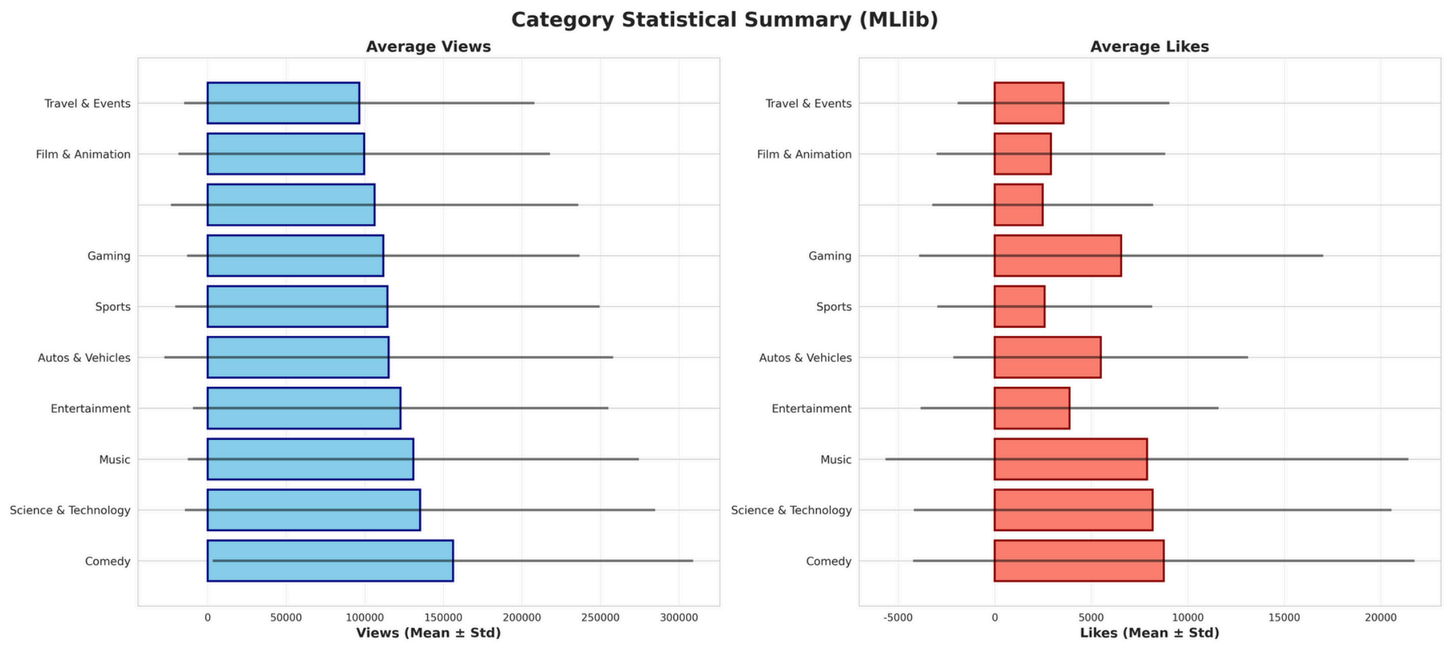
- **204** categories analyzed
- Strong engagement patterns identified
- Keyword impact quantified
- Pipeline successfully implemented

Deliverables:

✓ 12 high-quality visualizations
✓ Reproducible pipeline
✓ Complete source code package
✓ Deployment manual

Statistical Summary (MLlib Results)

- **Category Statistical Analysis**



Key Statistics:

- Mean views range: 96K - 208K
- High standard deviation in all categories
- Comedy shows highest engagement rate
- Entertainment has most content volume

Challenges & Solutions

- **Challenges Encountered & Solutions**

1. **Challenge:** Overlapping tools causing complexity
   Solution: Simplified to HDFS + Spark + Hive stack

2. **Challenge:** Slow query performance on CSV data
   Solution: Implemented ORC columnar format with partitioning
   Result: 10x speedup.

3. **Challenge:** Complex queries difficult to maintain
   Solution: Applied CTEs and window functions
   Result: Modular, readable SQL code

4. **Challenge:** Limited temporal data for trend analysis
   Solution: Acknowledged limitation, focused on available insights
   Lesson: Time series requires extended date ranges

5. **Challenge:** Memory errors during MLlib processing
   Solution: Sample 20% of data, adjusted executor memory
   Result: Stable execution

Technical Decisions:
✓ Batch over streaming (appropriate for historical analysis)
✓ Lambda Architecture batch layer
✓ Query optimization prioritized over model complexity

Conclusions & Future Work

- **Project Conclusions**

02

Achievements:
✓ Successfully implemented end-to-end big data pipeline
✓ Answered 4 research questions with data-driven insights
✓ Demonstrated scalable architecture (Lambda batch layer)
✓ Achieved efficient processing (5s for 4 queries)
✓ Created reproducible, well-documented system

03

Key Learnings:
- Simplicity beats complexity in architecture design
- Columnar storage critical for analytical workloads
- Monitoring metrics essential for optimization
- Data quality and coverage impact analysis depth

04

Impact:
→ Reproducible framework for YouTube trend analysis
→ Scalable to larger datasets (millions of records)
→ Applicable to other video platforms

Thank You